Understanding Mixture of Experts

From Sparse Routing to Modern MoE Language Models

A short technical handbook for AI engineers, developers, and serious learners

@techNmak

Handbook 07 - Understanding AI Series

ORIENTATION

What this handbook will make clear

A Mixture-of-Experts layer is easy to summarize as "route each token to a few experts." That sentence is correct, but it hides nearly every engineering and mathematical detail that makes modern MoE models interesting.

By the end of this handbook, you should be able to explain:

- what changes when a Transformer feed-forward layer becomes an MoE layer;
- how router logits, scores, Top-k selection, and expert weighting fit together;
- why total parameters, active parameters, FLOPs, weight memory, and wall-clock time are different quantities;
- why load imbalance appears, what auxiliary losses actually optimize, and why router stability is a separate problem;
- what expert capacity and token dropping mean, and why neither is fundamental to MoE;
- why distributed MoE requires token dispatch, expert-parallel communication, and efficient grouped expert computation;
- how designs such as Mixtral, DeepSeekMoE/DeepSeek-V3, and Qwen3 differ without changing the core MoE idea.

Prerequisites

You should know the basic Transformer block, the role of the feed-forward network, matrix multiplication, softmax, and the difference between training and inference. The earlier *Understanding Attention* handbook is useful context, but MoE routing is not an attention mechanism.

Scope

This handbook focuses on sparse expert layers in modern language-model Transformers. It covers enough expert parallelism to explain execution cost, but full distributed-systems design belongs in a separate systems handbook.

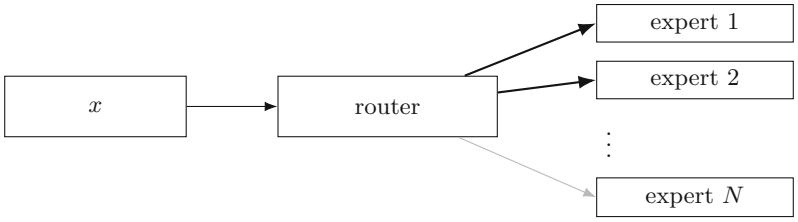
Research anchors: [1]-[4], [9]-[16]

Why Mixture of Experts exists

A dense neural-network layer applies the same learned parameters to every token that reaches it. Increasing the layer width or depth therefore increases both parameter capacity and the computation performed for every token.

Mixture of Experts introduces another scaling axis: *conditional computation*. The model can own a large bank of parameters while activating only a subset for a particular input. The central goal is not "free parameters." It is to decouple, to some degree, the amount of parameter capacity stored in the model from the amount of expert computation applied to one token.

For a token representation x, a sparse expert layer selects a small set of experts from a larger bank. If there are N experts but only $k \ll N$ are selected, the other experts need not execute for that token.



The sparse choice is input-dependent. Two adjacent tokens can use different experts, and the same token can use different experts at different layers because its hidden representation changes through the network.

Important. Sparse activation reduces which expert computations are executed. It does not automatically reduce the memory required to store all expert weights, nor does it guarantee lower wall-clock latency.

Research anchors: [2], [4], [7]

What an expert replaces

In a standard Transformer, the feed-forward sublayer is applied independently to every token after the attention sublayer. A simplified two-matrix feed-forward network can be written

$$\mathrm{FFN}(x) = W_2\,\phi(W_1 x),$$

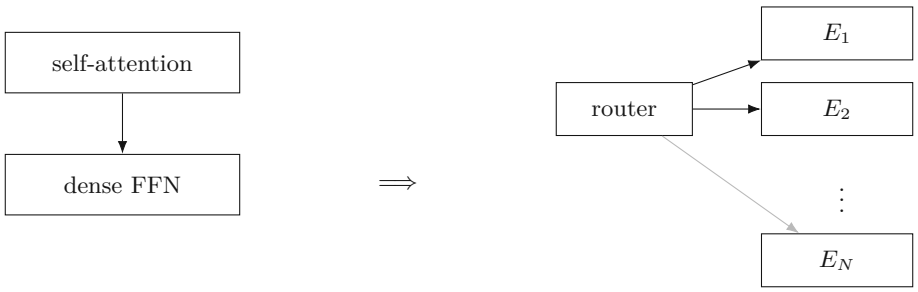
where $x \in \mathbb{R}^{d_{\text{model}}}$, W_1 expands the hidden dimension, ϕ is a nonlinearity, and W_2 projects back to the model dimension.

Modern LLMs often use gated variants such as SwiGLU, which involve an additional learned projection. The exact FFN formula is architecture-specific. The useful baseline is simpler: a dense Transformer has *one set of FFN parameters at that layer*, and every token uses that same set.

An MoE Transformer usually changes this sublayer, not the attention mechanism. Instead of one FFN, the layer owns several FFNs with the same input/output shape but different parameters:

$$E_1(x), E_2(x), \ldots, E_N(x).$$

A router decides which of them process each token.



Mental model. In modern sparse Transformer MoEs, an "expert" is usually a feed-forward subnetwork inside a layer. It is not a complete independent language model.

Research anchors: [3], [4], [9], [11]

One equation, sparse support

Let $E_i(x)$ be expert i and let $G_i(x)$ be the router's weight for that expert. A general mixture-of-experts layer can be written

$$\boxed{\mathrm{MoE}(x) = \sum_{i=1}^{N} G_i(x) E_i(x)}.$$

The equation itself does not require sparsity. The computational advantage appears when most entries of $G(x)$ are exactly zero, so only a small support set

$$\mathcal{S}(x) = \{i : G_i(x) \neq 0\}$$

needs to be evaluated.

For Top-k token-choice routing, $|\mathcal{S}(x)| = k$ under ordinary conditions. The output is then

$$y = \sum_{i \in \mathcal{S}(x)} g_i(x) E_i(x).$$

The experts must produce compatible output shapes so their contributions can be combined. They can, in principle, have different internal architectures, but large language models commonly use homogeneous FFN experts because regularity simplifies training and execution.

This formulation also explains what MoE does *not* change. The token representation still has the model's ordinary hidden dimension at the MoE boundary. Sparsity lives in parameter selection and expert execution, not in a smaller token vector.

Important. The same high-level equation covers many different routers. Softmax Top-k, sigmoid-based affinity, balanced assignment, and expert-choice methods differ in how $G(x)$ is constructed. The equation is the abstraction, not a specification of one implementation.

Research anchors: [2], [7], [9], [10], [11]

Where MoE sits in the block

A Transformer block usually alternates a token-mixing operation (attention) with a position-wise feed-forward operation. Sparse Transformer MoE models normally replace some or all of those dense FFN sublayers with expert banks.

A schematic pre-normalized block can be thought of as

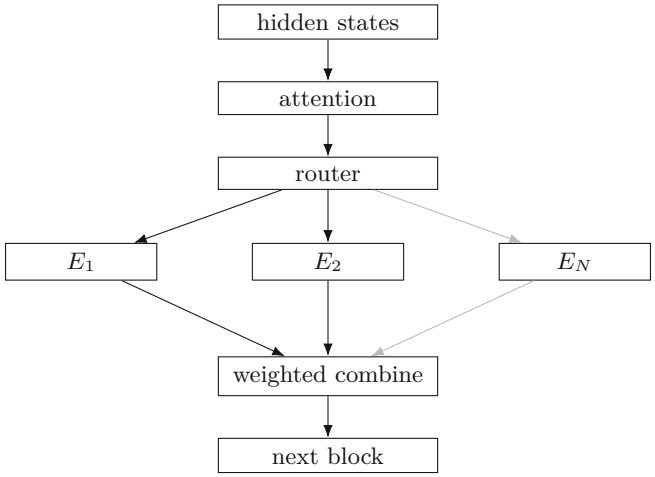
$$h' = h + \mathrm{Attention}(\mathrm{Norm}(h)),$$

$$h'' = h' + \mathrm{FFN}(\mathrm{Norm}(h')).$$

An MoE version changes the second computation to

$$h'' = h' + \mathrm{MoE}(\mathrm{Norm}(h')).$$

The exact normalization and residual ordering varies by model, so these equations are a structural sketch rather than a universal implementation.



GShard placed sparse MoE layers in a Transformer and scaled them across many devices. Switch Transformer later simplified token routing to one expert. Mixtral replaces the feed-forward blocks with sparse expert layers and routes each token to two experts.

Important. MoE and attention sparsity are separate ideas. A model can have dense attention and sparse FFNs, sparse attention and dense FFNs, or both.

Research anchors: [3], [4], [9]

From a token vector to expert scores

For the common linear router, take a token representation $x \in \mathbb{R}^d$ and a router matrix

$$W_r \in \mathbb{R}^{d \times N}.$$

Using row-vector notation, the router logits are

$$h = xW_r \in \mathbb{R}^N.$$

One logit is produced for each expert. In a softmax router,

$$p_i(x) = \frac{e^{h_i}}{\sum_{j=1}^{N} e^{h_j}}.$$

These values form a dense probability distribution before sparsification.

The router is small relative to a wide FFN expert, but it is not free. Its score computation grows with $d \times N$. Switch Transformer explicitly notes this router term while observing that the expert computation need not grow with the total number of experts when only one expert is selected.

Two terminology details matter.

- **Router logit**: the pre-normalization score produced by the routing network.
- **Gate/affinity weight**: the quantity used to rank and/or weight experts after applying the router's chosen transformation.

Those two are not always related by softmax. DeepSeek-V3, for example, computes routed-expert affinities with a sigmoid and normalizes selected affinities for combination.

Mental model. The router does not "understand expert names." It learns a mapping from the current hidden representation to numerical scores whose training pressure comes from the task loss and, depending on the design, balancing/stability objectives.

Research anchors: [2], [4], [11]

Selection, weighting, and the hard decision

After computing scores for all experts, token-choice routing keeps only a small number. For a softmax Top-k router, define

$$\mathcal{S}(x) = \mathrm{TopK}(p(x), k).$$

A generic output is

$$y = \sum_{i \in \mathcal{S}(x)} \tilde{p}_i(x) E_i(x).$$

What exactly is $\tilde{p}_i$? There is no single universal answer.

- Shazeer's sparse gate applies softmax after non-selected logits are set to $-\infty$.
- Switch uses the selected router probability as the gate value for its Top-1 expert.
- Mixtral describes softmax over Top-K logits.
- DeepSeek-V3 uses sigmoid affinities, selects with Top-K, then normalizes the selected affinities.

This is why a handbook should not write "Top-k always renormalizes the original softmax probabilities" as a definition.

How can a hard choice be trained?

The index selection itself is discrete. Within a region where the selected set does not change, however, selected gate values are differentiable functions of the router scores. Shazeer et al. explicitly note nonzero derivatives for selected experts when $k > 1$, and Switch notes that its gate value supplies a differentiable path even with Top-1 routing. Auxiliary router losses can provide additional gradient signals.

At selection boundaries the mapping is discontinuous. In practice, successful MoE training does not require pretending the Top-k index operation is a smooth function everywhere.

Research anchors: [2], [4], [9], [11]

From logits to two selected experts

Consider four experts and one token with router logits

$$h = [2.0,\ 1.2,\ 0.1,\ -0.4].$$

Applying softmax gives, to four decimals,

$$p \approx [0.5919,\ 0.2659,\ 0.0885,\ 0.0537].$$

For Top-2 routing, experts 1 and 2 are selected.

For this worked example only, suppose the implementation renormalizes the selected probabilities so they sum to one:

$$\tilde{p}_1 = \frac{0.5919}{0.5919 + 0.2659} \approx 0.6900,$$

$$\tilde{p}_2 = \frac{0.2659}{0.5919 + 0.2659} \approx 0.3100.$$

The sparse gate becomes

$$G(x) \approx [0.6900,\ 0.3100,\ 0,\ 0].$$

Expert	logit	dense softmax	selected weight
E_1	2.0	0.5919	0.6900
E_2	1.2	0.2659	0.3100
E_3	0.1	0.0885	0
E_4	-0.4	0.0537	0

The arithmetic shows two distinct operations: ranking experts and deciding how much each selected expert contributes.

Important. The renormalization step above is pedagogical, not a universal MoE rule. Always inspect the model's router definition before reproducing this calculation for a real architecture.

Research anchors: [2], [9], [11]

Combining the expert outputs

Continue the previous example with selected weights

$$\tilde{p}_1 \approx 0.6900, \qquad \tilde{p}_2 \approx 0.3100.$$

Suppose the two selected experts return

$$E_1(x) = [1.0, -0.5], \qquad E_2(x) = [2.0, 1.5].$$

Then the MoE output is

$$y = 0.6900E_1(x) + 0.3100E_2(x),$$

so

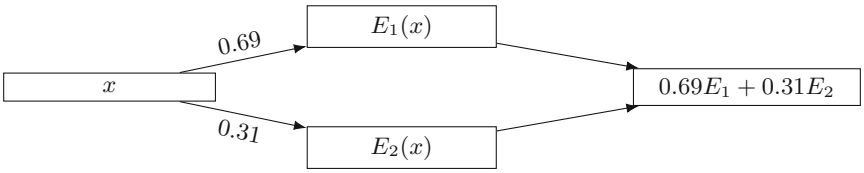
$$y \approx [1.3100,\ 0.1201].$$

The first coordinate is

$$0.6900(1.0) + 0.3100(2.0) = 1.3100,$$

and the second is

$$0.6900(-0.5) + 0.3100(1.5) \approx 0.1201.$$



This weighted combination is why Top-2 routing is not merely "pick two experts and average them." The router can modulate their contributions unequally.

In a Transformer, this expert result then participates in the block's ordinary residual/normalization structure according to that model's architecture.

Mental model. Routing chooses *where* computation happens. Expert functions compute candidate transformations. Gate weights determine how the selected transformations are combined.

Research anchors: [2], [9]

The assignment matrix matters

MoE execution becomes a systems problem when we route many tokens together. For T tokens and N experts, a binary routing map can be represented as

$$R \in \{0,1\}^{T \times N},$$

where $R_{t,i} = 1$ means token t is assigned to expert i.

For four tokens, four experts, and Top-2 routing, consider

$$R = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}.$$

Each row sums to 2 because each token uses two experts. Column sums give expert loads:

$$[3,\ 2,\ 2,\ 1].$$

There are $4 \times 2 = 8$ expert assignments in total, but the assignments are not balanced.

This simple matrix exposes several later concepts:

- **expert load**: the number of routed tokens in a column;
- **capacity**: how many assignments an implementation allows an expert to process;
- **dispatch**: regrouping token rows by the experts that need them;
- **combine**: returning processed expert outputs to the original token order.

When experts live on different devices, columns of this logical matrix correspond to work that may need to move across the network.

Important. A model may be mathematically sparse while being operationally inefficient if its assignment matrix creates tiny expert batches, severe imbalance, or expensive communication.

Research anchors: [2], [8], [16]

MoE predates the Transformer

Mixture-of-experts ideas are much older than modern LLMs. Jacobs, Jordan, Nowlan, and Hinton introduced *Adaptive Mixtures of Local Experts* in 1991: separate expert networks learned different subsets of training cases while a gating mechanism allocated cases among them.

The modern sparse-LLM lineage adds two crucial ingredients.

Sparse conditional computation

Shazeer et al. (2017) turned MoE into a general-purpose neural-network component with potentially thousands of feed-forward experts and a trainable sparse gate. Only a handful of experts were evaluated for each input. Their language models placed the MoE between recurrent layers rather than inside a Transformer.

Transformer-scale MoE

GShard then integrated sparsely gated MoE into Transformers and demonstrated automatic sharding of a multilingual translation model beyond 600B parameters. Switch Transformer simplified routing to a single selected expert and studied training stability, load balance, and capacity at large scale.

Year	Work	Durable idea
1991	Jacobs et al.	learned gating across local experts
2017	Shazeer et al.	sparse Top-k conditional computation
2020	GShard	Transformer MoE at very large distributed scale
2022	Switch	Top-1 routing and simplified sparse execution
2024+	modern LLM MoEs	new expert granularity, balancing, and serving designs

Mental model. MoE is not a trick invented for today's giant LLMs. Modern models are one stage in a longer evolution from learned modularity to sparse conditional computation at distributed scale.

Research anchors: [1]-[4]

The 2017 sparsely-gated formulation

Shazeer et al. proposed a noisy Top-k gate. In their notation, a dense pre-gate value is

$$H_i(x) = (xW_g)_i + \epsilon_i \text{ Softplus}((xW_{noise})_i),$$

with Gaussian noise ϵ_i. A Top-k operator keeps the largest k values and sets the rest to $-\infty$, then softmax produces the sparse gate:

$$G(x) = \text{Softmax}(\text{KeepTopK}(H(x), k)).$$

Thus non-selected experts receive exactly zero gate weight and do not need to execute for that input.

The noise served a training purpose in the original design, including support for load balancing and exploration. It should not be generalized into "all MoE routers add noise." Many later routers use different mechanisms.

The same paper highlighted two systems costs that are still relevant:

- **shrinking expert batches**: with batch size b, N experts, and k assignments per token, the average expert sees roughly kb/N token assignments before distributed aggregation;
- **network bandwidth**: if experts are distributed, token representations and expert outputs have to move to/from the devices holding those experts.

Those are not incidental implementation bugs. They follow directly from sparse, input-dependent routing.

Important. The original 2017 gate is foundational, but modern routers should be described from their own definitions rather than assumed to be noisy Top-k copies.

Research anchors: [2]

How sparse experts entered large Transformers

GShard used sparse MoE inside a Transformer and combined it with automatic sharding. Its reported system scaled a multilingual translation model beyond 600B parameters across 2048 TPU v3 accelerators. The lasting architectural point is more important than that headline number: expert parameters could be distributed across devices while only selected expert paths executed for each token.

Switch Transformer simplified the routing algorithm further. Instead of Top-2, each token was sent to the single expert with the highest router probability. The authors reported three practical motivations: less routing work, smaller expert capacity requirements, and reduced communication complexity.

For Top-1 Switch routing,

$$y = p_{i^*}(x)E_{i^*}(x), \qquad i^* = \arg\max_i p_i(x).$$

The selected probability still multiplies the expert output, so the router has a differentiable gate-value path even though the identity of i^* is a hard choice.

Switch also made expert capacity and token overflow central engineering concepts because its TPU-oriented implementation used statically bounded expert batches.

Important. "Top-1 is simpler" is not the same claim as "Top-1 is universally better." Later successful models use Top-2, Top-8, and other routing budgets. k is an architectural and systems choice.

Research anchors: [3], [4]

The routing budget changes more than one number

The number of selected experts k affects the expert path in several ways.

Routing	Per-token expert work	Typical consequence
Top-1	one expert path	simpler dispatch, smallest expert-work budget
Top-2	two expert paths	mixture of two transformations, more assignments
Top-k	k expert paths	higher expert work/communication as k grows

If expert widths are held fixed, moving from Top-1 to Top-2 approximately doubles the expert-FFN computation for that MoE layer. It does *not* necessarily double whole-model FLOPs because attention, embeddings, normalization, shared experts, and other layers remain.

Likewise, if total expert count N grows while k and expert shape stay fixed, total stored expert parameters can grow substantially without proportionally increasing the expert work for one token. But router scoring grows with N, the larger weight bank still has to be stored/sharded, and distributed execution can become harder.

Modern examples make the diversity concrete: Switch uses Top-1, Mixtral 8x7B uses Top-2, Qwen3-30B-A3B reports 128 experts with 8 activated, and DeepSeek-V3 selects 8 routed experts plus its shared-expert path.

Mental model. N controls the size of the expert bank. k controls how much of that bank a token uses. Neither number alone tells you latency, memory, or model quality.

Research anchors: [4], [9], [11], [14]

A static execution constraint, not the definition of MoE

Dynamic routing can send uneven numbers of tokens to different experts. Some implementations therefore give each expert a fixed processing capacity.

For the Top-1 Switch Transformer implementation, expert capacity is defined as

$$\boxed{C = \frac{T}{N} \times c_f}$$

where T is the number of tokens in the routing group, N is the number of experts, and c_f is the capacity factor.

At $c_f = 1$, total expert slots exactly equal the token count in this Top-1 setting. A factor above one creates buffer for uneven routing, but can introduce padding, extra memory, and extra work in statically shaped implementations.

This formula is often repeated too broadly. For general Top-k routing there are kT token-expert assignments, and frameworks/papers differ in how they define capacity, grouping, padding, and whether k is folded into the capacity formula. Some modern implementations avoid fixed per-expert capacity entirely.

Important. Expert capacity is an execution policy associated with particular routing/runtime designs. MoE mathematically requires sparse expert selection, not a fixed-capacity buffer.

Capacity also has a scope. It may be computed per batch, per local routing group, per expert-parallel rank, or another unit. That scope changes which imbalances are visible to the allocator.

Research anchors: [4], [7], [16]

What a capacity factor actually buys

Take a Top-1 Switch-style routing group with

$$T = 1024 \text{ tokens}, \qquad N = 16 \text{ experts}, \qquad c_f = 1.25.$$

The evenly balanced load would be

$$\frac{1024}{16} = 64 \text{ tokens/expert}.$$

The configured capacity is

$$C = 64 \times 1.25 = 80 \text{ tokens/expert}.$$

Across all experts, the runtime has room for

$$16 \times 80 = 1280$$

expert slots, even though only 1024 token assignments exist. That 25% extra slot budget is the buffer against imbalance.

Suppose expert 3 receives 95 tokens. Under the fixed-capacity Switch behavior, only 80 fit. The remaining

$$95 - 80 = 15$$

tokens overflow that expert's capacity. In Switch, overflowed tokens skip the expert computation and continue through the residual path.

Increasing c_f would reduce the chance of overflow, but increases the amount of statically provisioned expert work/memory. Decreasing it saves capacity but makes overflow easier.

Mental model. Capacity factor is a systems trade-off between spare expert slots and the consequences of imbalance. It is not a quality knob with a universal optimal value.

Important. This worked example is Top-1 and Switch-specific. Do not reuse $T/N \times c_f$ blindly for every Top-k MoE implementation.

Research anchors: [4]

Overflow behavior is a design choice

Older fixed-capacity MoE implementations made token dropping visible because static expert buffers could overflow. In Switch Transformer, an overflowed token does not receive that sparse expert computation and is passed onward through the residual connection.

That history sometimes becomes the false statement "MoE drops tokens." Modern evidence shows why this is wrong.

MegaBlocks

MegaBlocks reformulated expert computation as block-sparse operations that can efficiently handle unequal expert loads. Its design explicitly avoids the choice between dropping tokens and padding all experts to a fixed capacity.

DeepSeek-V3

The DeepSeek-V3 report states that its load balancing is effective enough that it does not drop tokens during training, and describes deployment strategies intended to avoid dropping during inference as well.

Current Megatron Core

Current NVIDIA Megatron Core documentation defines `moe_expert_capacity_factor=None` as no token dropping by default. Capacity-limited modes remain available when static shapes or other execution requirements make them useful.

Important. "Dropless" does not mean "costless." Variable expert loads still have to be represented, communicated, scheduled, and computed efficiently.

The durable distinction is:

$$\text{routing imbalance} \neq \text{mandatory token loss.}$$

A runtime can handle imbalance through dynamic buffers, sparse kernels, padding, capacity limits, or other strategies.

Research anchors: [4], [8], [11], [16]

Why load imbalance can reinforce itself

A trainable router is free to prefer some experts over others. Without counter-pressure, that preference can become extreme.

Shazeer et al. described a self-reinforcing failure mode: a few favored experts receive more examples, therefore receive more task-gradient updates, improve faster, and become even more attractive to the router. Other experts may be under-trained.

Load imbalance creates two different problems.

Optimization problem

If most data flows through a small subset of experts, the model is not effectively using the parameter capacity it owns. Under-used experts receive less learning signal from the main task.

Systems problem

When experts are on different devices, the step can be limited by the most heavily loaded expert/rank. Some devices wait while others process large token batches. Capacity-limited implementations may overflow or pad. Dynamic implementations still suffer stragglers and irregular batch sizes.

These two problems explain why load balancing is not merely a cosmetic preference for equal histograms.

Mental model. Sparse routing creates a feedback loop between *which expert learns* and *which expert is chosen*. Balancing methods try to keep that loop from collapsing onto too few experts without overwhelming the main language-model objective.

Important. Perfectly uniform routing is not itself the learning objective. The model should still be allowed to use experts differently when useful. The challenge is preventing pathological imbalance while preserving useful specialization.

Research anchors: [2], [4], [11]

Importance and load are not the same quantity

The 2017 sparsely gated MoE paper separated two notions that are easy to conflate.

For a batch X, it defined expert *importance* as the sum of gate values:

$$\text{Importance}(X) = \sum_{x \in X} G(x).$$

An auxiliary loss penalized the squared coefficient of variation of those expert importance values:

$$L_{\text{importance}} = w_{\text{importance}} \, \text{CV}(\text{Importance}(X))^2.$$

Here CV = standard deviation/mean.

Why was another loss needed? Equal total gate weight does not guarantee equal token counts. One expert could receive a few tokens with large weights while another receives many tokens with small weights. The hardware sees different numbers of token rows even if summed gate mass looks balanced.

Shazeer et al. therefore also introduced a load-oriented objective tied to expected expert assignment under their noisy gate. Its exact construction depends on that noisy Top-k formulation.

Important. This is a useful historical lesson: "balance the router" is underspecified. You must ask whether a method balances probability mass, hard assignment counts, device load, sequence-local load, or something else.

Later work simplified or replaced these objectives. Switch uses a single differentiable proxy involving hard assignment fractions and average router probabilities. DeepSeek-V3 uses dynamic routing biases for its main batch-level balancing pressure.

Research anchors: [2], [4], [11]

A differentiable pressure toward uniform expert use

For Top-1 routing, Switch Transformer defines

$$L_{\text{aux}} = \alpha N \sum_{i=1}^{N} f_i P_i,$$

where

$$f_i = \frac{1}{T} \sum_{x \in B} \mathbf{1}\{\arg\max p(x) = i\}$$

is the fraction of tokens actually dispatched to expert i, and

$$P_i = \frac{1}{T} \sum_{x \in B} p_i(x)$$

is the average router probability assigned to that expert.

The hard fraction f_i is not differentiable, but P_i is. The product couples the observed routing load to a differentiable probability signal.

Consider four experts with

$$f = [0.50, 0.25, 0.25, 0], \qquad P = [0.40, 0.30, 0.20, 0.10].$$

Then

$$N \sum_i f_i P_i = 4(0.20 + 0.075 + 0.05) = 1.30.$$

For perfectly uniform $f = P = [0.25, 0.25, 0.25, 0.25]$,

$$N \sum_i f_i P_i = 1.00.$$

So the weighted loss is 1.30α in the imbalanced example versus α in the uniform example.

Important. The uniform reference value is not zero. The factor N keeps the scale of the uniform case constant as expert count changes.

This loss was designed for Switch's Top-1 setting. Do not transplant the equation into another routing scheme without checking how that architecture defines assignments and balance.

Research anchors: [4]

Stability is a different problem from load balance

An MoE router contains exponentials and hard routing decisions, so large router logits can create numerical and training-instability problems. ST-MoE introduced the router z-loss

$$\boxed{L_z(x) = \frac{1}{B}\sum_{i=1}^{B}\left(\log\sum_{j=1}^{N} e^{x_j^{(i)}}\right)^2}$$

for a batch of B token router-logit vectors over N experts.

The loss penalizes large log-normalizers and encourages router logits to remain smaller in magnitude. ST-MoE reports that this improved training stability in its experiments without the quality loss seen with tighter update clipping.

The total training objective in that work includes the main cross-entropy loss, a load-balance term, and the router z-loss with separate coefficients:

$$L_{\text{total}} = L_{\text{CE}} + c_B L_B + c_z L_z.$$

This distinction matters:

- a **load-balancing loss** shapes how tokens are distributed across experts;
- the **router z-loss** constrains router-logit scale for stability.

They can coexist because they solve different problems.

Important. Calling z-loss "another load-balancing loss" erases its purpose. A router can be numerically well-behaved and still imbalanced, or balanced while still suffering unstable logit magnitudes.

Research anchors: [7]

Token choice is common, not mandatory

Top-k token-choice routing asks each token which experts it wants. Other work changes the assignment problem itself.

BASE Layers

BASE formulates token-to-expert allocation as a balanced linear assignment problem. Its design enforces equal numbers of tokens per expert and therefore avoids an auxiliary balancing loss in that formulation.

Expert Choice

Expert Choice reverses the direction: each expert selects its top-scoring tokens up to a fixed bucket size. This fixes per-expert capacity by construction, while a token can be selected by a variable number of experts.

Method	Who chooses?	Load property
Token-choice Top-k	token chooses k experts	expert loads can vary
BASE	global balanced assignment	equalized assignment by construction
Expert Choice	expert chooses top tokens	fixed expert bucket, variable experts/-token

These approaches illustrate a broader point: sparse MoE is a family of conditional-computation architectures, not one immutable router algorithm.

Important. Routing formulation changes semantics, optimization, and execution. An alternative that balances loads perfectly may impose different constraints on token coverage, causality, batching, or implementation. “Balanced” is not sufficient to declare one router universally superior.

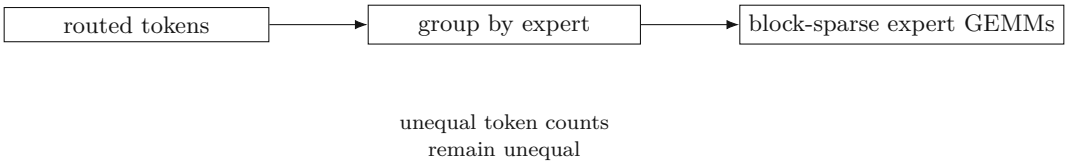
Research anchors: [5], [6]

Block-sparse kernels change the capacity trade-off

A dynamic MoE naturally gives different experts different token counts. Dense accelerator kernels prefer regular, sufficiently large matrices. That mismatch historically pushed runtimes toward fixed capacities, padding, or token dropping.

MegaBlocks attacks the mismatch at the kernel level. It represents expert FFN computation with block-sparse matrix operations that can handle variable expert loads efficiently. The MLSys paper's central systems claim is that this removes the forced choice between dropping overflow tokens and padding experts to a common capacity.

The resulting mental model is different from Switch's static buffers:



No mathematical change to MoE is required. The change is how sparse, imbalanced expert batches are mapped to GPU computation.

Current production frameworks expose both families of behavior. Megatron Core supports capacity factors and token-drop policies, but its default capacity factor is currently `None`, documented as no token dropping.

Mental model. MoE algorithm design and MoE kernel design are coupled. A router can be mathematically attractive yet expensive if the resulting sparse workload maps poorly to hardware.

Research anchors: [8], [16]

DeepSeekMoE changes the expert bank itself

DeepSeekMoE proposes two architectural changes aimed at expert specialization.

Fine-grained expert segmentation

Starting from N experts with K active, the paper conceptually splits experts more finely into mN smaller experts and activates mK. Because each smaller expert contains fewer parameters, the design can expose more combinations while controlling expert parameter and computation budgets.

A purely combinatorial illustration shows why the routing space expands. Choosing 2 of 8 experts gives

$$\binom{8}{2} = 28$$

unordered expert sets. Splitting to 16 smaller experts and choosing 4 gives

$$\binom{16}{4} = 1820.$$

This does *not* mean "65x more intelligence." It only illustrates the much larger set of possible expert combinations.

Shared expert isolation

DeepSeekMoE also reserves some experts as shared experts that are applied broadly, while routed experts remain sparse. The motivation is to let shared paths capture common knowledge and reduce redundancy pressure among routed experts.

DeepSeek-V3 uses this family of design: shared expert computation is added alongside selected routed-expert outputs.

Important. Shared experts change active-compute accounting. A token can execute its routed experts *and* shared expert paths, so "Top-8" is not necessarily the complete FFN computation story.

Research anchors: [10], [11]

What DeepSeek-V3 actually means by “loss-free”

DeepSeek-V3 uses sigmoid token-to-expert affinities $s_{i,t}$ for its routed experts. In its main load-balancing mechanism, each expert has a routing bias b_i. Top-K selection uses

$$s_{i,t} + b_i,$$

but the expert's output weight is derived from the original affinity $s_{i,t}$, not the biased score.

At the end of a training step, the bias is decreased for overloaded experts and increased for underloaded experts. The bias therefore changes *who gets selected* without directly changing the affinity value multiplied into the expert output.

This is the sense in which the paper calls its main batch-level method *auxiliary-loss-free*: load pressure is applied through dynamic routing biases rather than a conventional batch-level auxiliary loss term that competes with the language-model objective.

There is an important qualification. DeepSeek-V3 also states that it uses a complementary *sequence-wise* balance loss with an extremely small coefficient to prevent extreme imbalance inside an individual sequence.

The report additionally states that DeepSeek-V3 does not drop tokens during training, and describes node-limited routing to constrain communication.

Important. “DeepSeek-V3 uses no balancing loss” is too broad. The accurate statement is that its primary batch-level balancing strategy is bias-based and auxiliary-loss-free, while a small complementary sequence-wise balance loss remains.

Research anchors: [11]

The most abused MoE comparison

MoE models are often described with two parameter counts.

Total parameters

All model weights stored across dense layers, attention, embeddings, routers, shared experts, and the complete routed-expert bank. These weights matter for checkpoint size, memory capacity, sharding, loading, and communication.

Active parameters per token

The subset of parameters whose computations are used for one token's forward pass, according to the architecture's accounting. In an MoE layer this includes selected expert paths and may include shared experts. Dense parts of the model remain active too.

The difference is easiest to see in Mixtral 8x7B. Its paper reports 47B total parameters accessible to the model and 13B active parameters per token. The paper explicitly warns that active-parameter analysis does not capture weight-memory cost or hardware utilization.

A further subtlety appears at batch level. Token A may activate experts 1 and 4 while token B activates 2 and 7. Across a large batch, many or all experts can be used even though each individual token activates only a few.

Important. Active parameters are not the model's memory footprint, and they are not a universal proxy for latency. Kernel efficiency, weight movement, routing, communication, batch shape, precision, and hardware all matter.

Mental model. "Sparse per token" does not mean "most model weights can be absent from the serving system."

Research anchors: [9], [11], [14], [16]

More stored expert capacity without more expert work in a toy layer

Use a deliberately simple two-matrix FFN so the arithmetic is transparent. Ignore biases.

A dense FFN with

$$d_{\text{model}} = 1024, \qquad d_{\text{ff}} = 4096$$

has

$$2 \times 1024 \times 4096 = 8{,}388{,}608$$

weight parameters.

Now create an MoE with 8 smaller experts. Each expert has $d_{\text{ff}} = 2048$, so one expert has

$$2 \times 1024 \times 2048 = 4{,}194{,}304$$

weights. The full expert bank has

$$8 \times 4{,}194{,}304 = 33{,}554{,}432$$

expert weights.

If each token uses Top-2 routing, the selected expert paths touch

$$2 \times 4{,}194{,}304 = 8{,}388{,}608$$

expert weights, equal to the dense toy FFN's weight count for its two matrix multiplies. The router matrix adds only

$$1024 \times 8 = 8192$$

weights in this toy setup.

So this constructed example has 4x the stored expert-bank parameters while matching the dense baseline's idealized expert-matrix parameter count per token.

Important. This is an arithmetic illustration, not a performance guarantee. Real LLM FFNs may use gated three-projection designs, shared experts, different widths, padding, quantization, communication, and fused kernels. Equal parameter-touch counts do not imply equal runtime.

Research anchors: [2], [4], [9]

The router exists in both phases, but the problems differ

During training, many tokens are routed together and gradients flow through the selected expert computations and router weights. An expert receives main-task gradient only from tokens that use it on that step. Balancing and stability objectives, when present, add router-specific training signals.

This creates batch-level dynamics: expert utilization affects how much each expert learns, and the current router affects the data distribution seen by each expert.

During inference, the task is different. There is no backward pass and training-only auxiliary losses are not optimized, but the model still has to compute routing decisions, dispatch token states, execute selected experts, and combine outputs.

For autoregressive decode, each sequence typically contributes one new token per decode step. Without batching many requests, an individual expert can receive very few tokens at a time, making expert GEMMs small and inefficient. Large serving batches can improve arithmetic intensity but also make routing/load patterns more important.

Routing is also layer-specific. A generated token can select one set of experts at an early layer and another set later because its hidden representation changes.

Important. "MoE training is sparse, so inference is automatically fast" is not a valid deduction. Sparse arithmetic helps only if the serving implementation can exploit the resulting routing pattern efficiently.

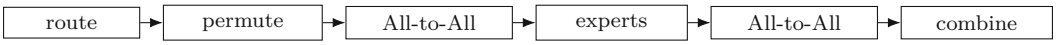
Research anchors: [2], [9], [11], [16]

Dispatch, compute, combine

A large expert bank may not fit, or may not be efficient, on one GPU. Expert parallelism partitions experts across devices. The router can therefore select an expert that is physically remote from the token's current device.

A common execution pattern is:

1. compute routing indices and weights;
2. permute/group token states by destination expert;
3. communicate tokens to devices that host those experts;
4. execute expert FFNs, often as grouped or fused GEMMs;
5. communicate expert outputs back;
6. unpermute and combine outputs in original token order.



Current Megatron Core documents All-to-All, AllGather, and flexible token-dispatch backends. Its All-to-All dispatcher explicitly performs dispatch to expert ranks, expert-major reordering, then a combine communication and unpermutation.

Important. Expert parallelism is not the definition of MoE. It is a distributed execution strategy for the expert bank. A small MoE can run on one device with no inter-GPU expert communication.

Research anchors: [3], [11], [16]

Why fewer active parameters can still run badly

Sparse expert computation changes the performance model rather than deleting it. Several costs compete.

Expert GEMM size

With many experts, tokens are split into smaller per-expert batches. Tiny or irregular matrices can underutilize accelerators. The 2017 MoE paper called this the shrinking-batch problem.

Communication

Expert parallelism moves token states to expert devices and outputs back. All-to-All communication can become comparable to expert computation if the compute/communication ratio is poor.

Weight movement

Even when only a few experts are used per token, a batch can touch many experts. Weight residency and memory bandwidth therefore remain important, especially at low batch sizes.

Load imbalance

The slowest expert/rank can determine the exposed step time if work is not effectively overlapped.

Routing and permutation overhead

Score computation, Top-k, metadata construction, sorting/permutation, and combine operations all consume time.

Modern systems respond with grouped GEMMs, block-sparse kernels, topology-aware routing, fused dispatch, and computation/communication overlap. DeepSeek-V3, for example, explicitly co-designs routing with cross-node communication and limits the number of nodes a token can contact.

Mental model. Active parameter count describes the mathematical path length through weights. Performance depends on whether hardware can execute that path with large, local, well-balanced operations.

Research anchors: [2], [8], [9], [11], [16]

Do experts become “math experts” and “code experts”?

It is tempting to turn MoE into a human story: one expert learns mathematics, another learns code, another learns French. Research does not justify that as a general mental model.

The original sparsely-gated MoE reported specialization patterns in its recurrent language models. But specialization can look different across architectures and layers.

Mixtral performed a routing analysis across domains from The Pile. The authors reported no obvious domain-level allocation pattern for most datasets: ArXiv, PubMed, philosophy, and others had similar expert-assignment distributions. They did observe more structured syntactic behavior, such as recurring assignments for indentation or certain token patterns, and substantial locality in some layers.

ST-MoE also found that specialization depended on context. Its analysis reports specialization patterns in encoder experts while decoder experts did not exhibit the same clear behavior, and multilingual specialization did not reduce to simple “one language per expert” partitions.

This makes three concepts worth separating:

- **routing frequency**: which experts get selected;
- **functional specialization**: whether experts perform measurably different transformations;
- **human-interpretable specialization**: whether those differences map cleanly to labels such as “math.”

Important. A colorful expert-routing plot is evidence about selection patterns, not automatically evidence that an expert stores a clean semantic domain.

Research anchors: [2], [7], [9]

Three designs, three different MoE choices

Modern open models make it clear that "MoE" does not imply one expert count or routing rule.

Model	Total / active	Expert routing	Selected architecture facts
Mixtral 8x7B	47B / 13B	8 experts, Top-2	FFN experts, softmax over Top-2 logits
DeepSeek-V3	671B / 37B	256 routed, Top-8	fine-grained routed experts + shared expert, sigmoid affinity
Qwen3-30B-A3B	30.5B / 3.3B	128 experts, Top-8	official model card reports 48 layers and 8 activated experts

These headline parameter counts come from each model's authors and should be interpreted within that architecture's accounting. They are useful descriptions, not directly comparable latency measurements.

Mixtral's paper explicitly notes that active-parameter analysis does not include memory cost and hardware utilization. DeepSeek-V3 couples its expert architecture to node-limited routing and custom communication. Qwen3's model card reports a much larger expert bank than Mixtral with eight active experts per token.

Important. There is no monotonic rule that "more experts" or "fewer active parameters" produces a faster or better model. Expert size, dense backbone, routing, precision, parallelism, and implementation all change the comparison.

Research anchors: [9], [11], [14]

A pedagogical Top-2 token-choice router

The following PyTorch-style sketch exposes the conceptual stages. It is original pedagogical code, not an efficient production MoE implementation.

```
# x: [tokens, d_model]
logits = x @ router_weight          # [tokens, experts]
probs = logits.softmax(dim=-1)

top_p, top_i = probs.topk(k=2, dim=-1)
top_p = top_p / top_p.sum(dim=-1, keepdim=True)

out = torch.zeros_like(x)
for slot in range(2):
    for expert_id, expert in enumerate(experts):
        mask = (top_i[:, slot] == expert_id)
        if mask.any():
            y = expert(x[mask])
            out[mask] += top_p[mask, slot, None] * y
```

It makes the mathematical steps visible:

$$\text{scores} \rightarrow \text{Top-2 indices} \rightarrow \text{expert calls} \rightarrow \text{weighted combine.}$$

What the code deliberately hides is most of the hard engineering: capacity policies, stable routing precision, distributed dispatch, token permutation, grouped GEMMs, communication overlap, shared experts, load-balancing losses, training noise, and specialized kernels.

The nested Python loops would be disastrous at scale. Production systems group many tokens for the same expert and invoke batched/fused expert computation instead.

Important. This example also renormalizes the selected softmax probabilities. That is one valid pedagogical convention, not the universal router definition for every model.

Research anchors: [2], [9], [16]

Bugs that still look plausible

1. Treating Top-k indices as differentiable probabilities.

The membership decision is discrete. Gradients flow through selected weights and auxiliary objectives according to the router design, not through an imaginary smooth argmax.

2. Assuming all routers use softmax.

DeepSeek-V3 uses sigmoid affinities for routed experts. Router normalization and weighting must match the model.

3. Normalizing over the wrong set.

Softmax over all experts followed by masking, softmax after Top-k masking, and normalization only among selected sigmoid affinities are different computations.

4. Confusing token count with assignment count.

Top-k creates up to kT token-expert assignments from T tokens. Capacity formulas must match the router/runtime convention.

5. Losing original token order after dispatch.

Expert-major execution requires metadata sufficient to combine outputs back into the correct token rows and routing slots.

6. Using one expert's load statistics at the wrong scope.

Per-device, per-group, per-sequence, and global-batch balance can differ.

7. Assuming active parameters equal resident weights.

Serving may need the whole sharded expert bank even though a token uses only a subset.

8. Comparing MoE speed without naming hardware and batch shape.

Sparse execution efficiency is highly sensitive to expert batch size, interconnect, precision, and kernels.

Research anchors: [4], [7], [11], [16]

Shortcuts that create the wrong mental model

"An expert is a whole LLM."	Usually false in modern Transformer MoEs. Experts are typically FFN/MLP subnetworks inside selected layers.
"MoE runs only a tiny model."	A token activates sparse expert paths, but dense/shared parameters still run and the full expert bank still exists.
"100B total and 10B active means 10B memory."	Active parameters are not checkpoint or resident-weight memory.
"MoE always uses Top-2."	Switch is Top-1, Mixtral Top-2, Qwen3 and DeepSeek-V3 use larger routing budgets.
"MoE always drops tokens."	No. Dropping is one capacity policy. MegaBlocks and current dropless runtimes show otherwise.
"Load balancing means every expert should be identical."	Balance concerns utilization/load. Useful functional differences can still emerge.
"Each expert learns one domain."	Routing analyses do not support this as a universal interpretation.
"More experts makes inference faster."	More experts increases capacity but can worsen routing, weight movement, and communication.
"MoE sparsity is sparse attention."	MoE sparsifies parameter/expert activation. Sparse attention changes query-key connectivity.

Research anchors: [4], [8], [9], [11], [14]

Mixture of Experts in one chain

A modern sparse Transformer MoE can be remembered as nine linked ideas.

1. **Start with the FFN.** The expert bank usually replaces a dense feed-forward sublayer, not attention.
2. **Score.** A router maps each token representation to expert scores or affinities.
3. **Select.** A sparse routing rule chooses a small subset of experts.
4. **Compute.** Only selected routed experts execute for that token, plus any shared paths the architecture defines.
5. **Combine.** Router-derived weights merge selected expert outputs back into one token representation.
6. **Balance.** Training must prevent pathological expert under-use/over-use without letting balancing pressure dominate the task objective.
7. **Stabilize.** Router numerical stability is distinct from load balance; techniques such as z-loss target a different failure mode.
8. **Dispatch.** In expert-parallel systems, token states move to expert devices and outputs return, often through All-to-All communication.
9. **Measure the right resource.** Total parameters, active parameters, FLOPs, memory capacity, memory traffic, communication, and wall-clock time are not interchangeable.

MoE's durable promise is conditional computation: increase the parameter capacity available to the model without forcing every token through every expert parameter. Its durable difficulty is that conditional computation produces dynamic, imbalanced, distributed work that dense hardware does not execute automatically well.

If those two statements stay together, most MoE claims become easier to evaluate. The architecture is neither "a giant model for the price of a tiny one" nor merely "several FFNs with a router." It is a co-design problem spanning optimization, numerical stability, sparse computation, memory, communication, and model architecture.

Research anchors: [1]-[16]

Primary papers and current implementation sources

1. R. A. Jacobs, M. I. Jordan, S. J. Nowlan, G. E. Hinton. *Adaptive Mixtures of Local Experts.* Neural Computation 3(1):79-87, 1991. DOI: 10.1162/neco.1991.3.1.79.
2. N. Shazeer et al. *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer.* arXiv:1701.06538, 2017.
3. D. Lepikhin et al. *GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding.* arXiv:2006.16668, 2020; ICLR 2021.
4. W. Fedus, B. Zoph, N. Shazeer. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity.* JMLR 23(120):1-39, 2022.
5. M. Lewis et al. *BASE Layers: Simplifying Training of Large, Sparse Models.* arXiv:2103.16716, 2021.
6. Y. Zhou et al. *Mixture-of-Experts with Expert Choice Routing.* NeurIPS 2022. arXiv:2202.09368.
7. B. Zoph et al. *ST-MoE: Designing Stable and Transferable Sparse Expert Models.* arXiv:2202.08906, 2022.
8. T. Gale, D. Narayanan, C. Young, M. Zaharia. *MegaBlocks: Efficient Sparse Training with Mixture-of-Experts.* Proceedings of MLSys, Vol. 5, 2023. arXiv:2211.15841.
9. A. Q. Jiang et al. *Mixtral of Experts.* arXiv:2401.04088, 2024.
10. D. Dai et al. *DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models.* arXiv:2401.06066, 2024.
11. DeepSeek-AI. *DeepSeek-V3 Technical Report.* arXiv:2412.19437v2, 2025; official DeepSeek-V3 repository/configuration consulted for architecture details.
12. N. Muennighoff et al. *OLMoE: Open Mixture-of-Experts Language Models.* arXiv:2409.02060, 2024.
13. Qwen Team. *Qwen3 Technical Report.* arXiv:2505.09388, 2025.
14. Qwen Team. *Qwen3-30B-A3B official model card.* Current official model metadata consulted September 2026 for total/activated parameters and expert counts.
15. Mistral AI. *Mixtral 8x7B model release/source materials.* Used only where consistent with the primary Mixtral paper.
16. NVIDIA Megatron Core documentation. *MoE token dispatchers, expert parallelism, router/capacity configuration.* Current documentation consulted September 2026.